nature

SUPPLEMENTARY INFORMATION

https://doi.org/10.1038/s41586-019-0980-2

In the format provided by the authors and unedited.

Supervised learning with quantum-enhanced feature spaces

Vojtěch Havlíček[1,2], Antonio D. Córcoles[1]*, Kristan Temme[1]*, Aram W. Harrow[3], Abhinav Kandala[1], Jerry M. Chow[1] & Jay M. Gambetta[1]

[1]IBM T. J. Watson Research Center, Yorktown Heights, NY, USA. [2]Department of Computer Science, University of Oxford, Wolfson Building, Parks Road, Oxford, UK. [3]Center for Theoretical Physics, Massachusetts Institute of Technology, Cambridge, MA, USA. *e-mail: adcorcol@us.ibm.com; kptemme@ibm.com

CONTENTS

I. Classification problems 1

II. Description of the algorithms 2
A. Quantum variational classification 2
B. Quantum kernel estimation 3

III. The relationship of variational quantum classifiers to support vector machines 5
A. Support vector machines (SVMs) 5
B. Variational circuit classifiers 7

IV. Encoding of the data using a suitable feature map 8
A. Non-trivial feature map with entanglement 9

V. Quantum variational classification 11

VI. Choosing the cost-function for the circuit optimization 11
A. Binary label classification 12
B. Multi label classification 15

VII. Quantum kernel estimation 15

VIII. Device parameters 17
A. Gate characterization 17
B. Readout correction 18

IX. Error mitigation and additional experimental results 18
A. Results for additional data sets 19

References 21

I. CLASSIFICATION PROBLEMS

Consider a classification task on a set $C = \{1, 2 \dots c\}$ of c classes (labels) in a supervised learning scenario. In such settings, we are given a training set T and a test set S, both are assumed to be labeled by a map $m : T \cup S \to C$ unknown to the programmer. Both sets S and T are provided to the programmer, but the programmer only receives the labels of the training set. So, formally, the programmer has only access to a restriction $m_{|T}$ of the indexing map m:

$$m_{|T} : T \to C, \text{ s.t.: } m_{|T}(\boldsymbol{t}) = m(\boldsymbol{t}), \ \forall \boldsymbol{t} \in T.$$

It is the programmer's goal to use the knowledge of $m_{|T}$ to infer an indexing map $\tilde{m} : S \to C$ over the set S, such that $m(\boldsymbol{s}) = \tilde{m}(\boldsymbol{s})$ with high probability for any $\boldsymbol{s} \in S$. The accuracy of the approximation to the map is quantified by a classification success rate, proportional to the number of collisions of m and $\tilde{m}$:

$$\nu_{succ.} = \frac{|\{\boldsymbol{s} \in S | m(\boldsymbol{s}) = \tilde{m}(\boldsymbol{s})\}|}{|S|}.$$

For such a learning task to be meaningful it is assumed that there is a correlation in output of the indexing map m over the sets S and T. For that reason, we assume that both sets could in principle be constructed by drawing the S and T sample sets from a family of d-dimensional distributions $\left\{p_c : \Omega \subset \mathbb{R}^d \to \mathbb{R}\right\}_{c \in C}$ and labeling the outputs according to the distribution. It is assumed that the hypothetical classification function m to be learned is constructed this way. The programmer, however, does not have access to these distributions and the labeling function directly. She is only provided with a large, but finite, number of samples and the matching labels.

The conventional approach to this problem is to construct a family of classically computable function $\tilde{m} : \langle \boldsymbol{\theta}, S \rangle \to C$, indexed by a set of parameters $\boldsymbol{\theta}$. These weights are then inferred from $m_{|T}$ by an optimization procedure on a classical cost function. Here, we consider a scenario where the whole, or parts of the classification protocol m, are generated on a quantum computer.

II. DESCRIPTION OF THE ALGORITHMS

We consider two different learning schemes. The first is referred to as "Quantum variational classification", the second is referred to "Quantum kernel estimation". When binary classification problems are considered, both schemes construct a separating hyperplane in the state space of n qubits, c.f. section III B. The first scheme is also able to operate in a multi label setting. The classical data is mapped to this space with $\dim = 4^n$ using a unitary circuit family that depends non-linearly on a datum, starting from the reference state $|0\rangle\langle 0|^n$.

A. Quantum variational classification

For our first classification approach we design a variational algorithm which exploits the large dimensional Hilbert space of our quantum processor to find an optimal cutting hyperplane in a similar vein as Support Vector Machines (SVM) do. The algorithm consists of two main parts: a training stage, c.f. algorithm 3, and a classification stage c.f. algorithm 4. For the training stage, a set of labeled data points is provided, on which the algorithm is performed. For the classification stage, we take a different set of data points and run the optimized classifying circuit on them without any label input. Then we compare the label of each data point to the output of the classifier to obtain a success ratio for the data set. For both the training and the classification stages, the quantum circuit that implements the algorithm comprises three main parts: the encoding of the feature map, c.f. section IV A, Fig 2, the variational optimization, c.f. section V, Fig 3 and the measurement, c.f. section VI. These steps are combined in to the circuit depicted in Fig 1.

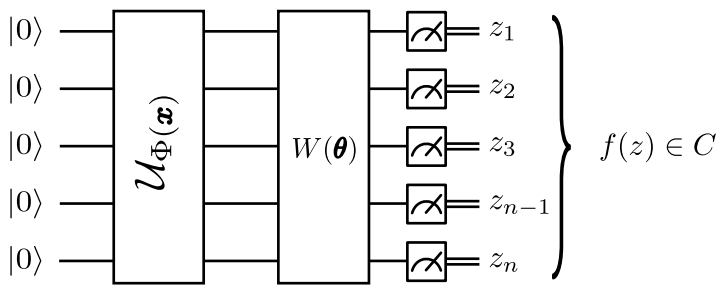

FIG. 1. Quantum variational classification: The circuit takes a references state, $|0\rangle^n$, applies the unitary $\mathcal{U}_{\Phi(\boldsymbol{x})}$ followed by the variational unitary $W(\boldsymbol{\theta})$ and applies a measurement in the Z-basis. The resulting bit string $z \in \{0,1\}^n$ is then mapped to a label in C. This circuit is re - run multiple times and sampled to estimate the expectation value $p_y = \langle \Phi(\boldsymbol{x}) | W^\dagger(\boldsymbol{\theta}) M_y W(\boldsymbol{\theta}) | \Phi(\boldsymbol{x}) \rangle$ for the labels $y \in C$. In the experiment we consider $C = \{+1, -1\}$.

We state the cost-function for the binary classification problem. The more general case is discussed in section VI B. During the training we optimize the parameters $\boldsymbol{\theta}$ of the variational circuit $W(\boldsymbol{\theta})$ and possibly a bias parameter $b \in [-1, +1]$. Both the feature map circuit $\mathcal{U}_\Phi$, as well as well as the function $f : \{0,1\}^n \to C$ are fixed and chosen by the programmer. For the optimization, we need to define a cost-function. This choice is discussed in section VI. We define the empirical risk $R_{\text{emp}}(\boldsymbol{\theta})$ in terms of the the error probability $\Pr(\tilde{m}(\boldsymbol{x}) \neq m(\boldsymbol{x}))$ of assigning the incorrect label, based on the decision rule $\tilde{m}(\boldsymbol{s}) = \text{argmax}_y\{\hat{p}_y(\boldsymbol{s})\}$ after R shots, averaged over the samples in the training set T. In the binary case this decision rule simplifies to $\tilde{m}(\boldsymbol{x}) = y$ whenever $\hat{p}_y(\boldsymbol{x}) > \hat{p}_{-y}(\boldsymbol{x}) - yb$.

$$R_{\text{emp}}(\boldsymbol{\theta}) = \frac{1}{|T|} \sum_{\boldsymbol{x} \in T} \Pr\left(\tilde{m}(\boldsymbol{x}) \neq m(\boldsymbol{x})\right) .$$

For the binary problem, the error probability of assigning the wrong label is given by the binomial cumulative density function (CDF) of the empirical distribution $\hat{p}_y(\boldsymbol{x})$, c.f. section VI A. The binomial CDF can be approximated for a large number of samples (shots) $R \gg 1$ by a sigmoid function $\text{sig}(x) = (1 + e^{-x})^{-1}$. The probability that the label $m(\boldsymbol{x}) = y$ is assigned incorrectly is approximated by

$$\Pr\left(\tilde{m}(\boldsymbol{x}) \neq m(\boldsymbol{x})\right) \approx \text{sig}\left(\frac{\sqrt{R}\left(\frac{1}{2} - \left(\hat{p}_y(\boldsymbol{x}) + \frac{yb}{2}\right)\right)}{\sqrt{2(1 - \hat{p}_y(\boldsymbol{x}))\hat{p}_y(\boldsymbol{x})}} \right) .$$

The training phase consists of the following steps:

Algorithm 1 Quantum variational classification: the training phase

Input Labeled training samples $T = \{\boldsymbol{x} \in \Omega \subset \mathbb{R}^n\} \times \{y \in C\}$, Optimization routine,
Parameters Number of measurement shots R, and initial parameter $\boldsymbol{\theta}_0$.
Calibrate the quantum hardware to generate short depth trial circuits.
Set initial values of the variational parameters $\boldsymbol{\theta} = \boldsymbol{\theta}_0$ for the short-depth circuit $W(\boldsymbol{\theta})$
while Optimization (e.g. SPSA) of $R_{\text{emp}}(\boldsymbol{\theta})$ has not converged **do**
  for $i = 1$ `to` $|T|$ **do**
    Set the counter $\mathrm{r}_y = 0$ for every $y \in C$.
    for *shot* $= 1$ `to` R **do**
      Use $\mathcal{U}_{\Phi(\boldsymbol{x}_i)}$ to prepare initial feature-map state $|\,\Phi(\boldsymbol{x}_i)\rangle\langle\Phi(\boldsymbol{x}_i)\,|$
      Apply discriminator circuit $W(\boldsymbol{\theta})$ to the initial feature-map state .
      Apply $|C|$ - outcome measurement $\{M_y\}_{y\in C}$
      Record measurement outcome label y by setting $\mathrm{r}_y \to \mathrm{r}_y + 1$
    end for
    Construct empirical distribution $\hat{p}_y(\boldsymbol{x}_i) = \mathrm{r}_y R^{-1}$.
    Evaluate $\Pr\left(\tilde{m}(\boldsymbol{x}_i) \neq y_i | m(\boldsymbol{x}) = y_i\right)$ with $\hat{p}_y(\boldsymbol{x}_i)$ and y_i
    Add contribution $\Pr\left(\tilde{m}(\boldsymbol{x}_i) \neq y_i | m(\boldsymbol{x}) = y_i\right)$ to cost function $R_{\text{emp}}(\boldsymbol{\theta})$.
  end for
  Use optimization routine to propose new $\boldsymbol{\theta}$ with information from $R_{\text{emp}}(\boldsymbol{\theta})$
end while
return the final parameter $\boldsymbol{\theta}^*$ and value of the cost function $R_{\text{emp}}(\boldsymbol{\theta}^*)$

The classification can be applied when the training phase is complete. The optimal parameters are used to decide the correct label for new input data. Again, the same circuit Fig 1 is applied, however, this time the parameters are fixed and and the outcomes are combined to determine the label which is reported as output of the classifier.

Algorithm 2 Quantum variational classification: the classification phase

Input An unlabeled sample from the test set $\boldsymbol{s} \in S$, optimal parameters $\boldsymbol{\theta}^*$ for the discriminator circuit.
Parameters Number of measurement shots R
Calibrate the quantum hardware to generate short depth trial circuits.
Set the counter $\mathrm{r}_y = 0$ for every $y \in C$.
for *shot* $= 1$ `to` R **do**
  Use $\mathcal{U}_{\Phi(\boldsymbol{s})}$ to prepare initial feature-map state $|\,\Phi(\boldsymbol{s})\rangle\langle\Phi(\boldsymbol{s})\,|$
  Apply optimal discriminator circuit $W(\boldsymbol{\theta^*})$ to the initial feature-map state .
  Apply $|C|$ - outcome measurement $\{M_y\}_{y\in C}$
  Record measurement outcome label y by setting $\mathrm{r}_y \to \mathrm{r}_y + 1$
end for
Construct empirical distribution $\hat{p}_y(\boldsymbol{s}) = \mathrm{r}_y R^{-1}$.
Set label $= \mathrm{argmax}_y\{\hat{p}_y(\boldsymbol{s})\}$
return label

B. Quantum kernel estimation

For the second classification protocol, we restrict ourselves to the binary label case, with $C = \{+1, -1\}$. Here it will be convenient to write $T = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t\}$ with $t = |T|$; also let $y_i = m(\boldsymbol{x}_i)$ be the corresponding label. In this protocol we only use the quantum computer to estimate the $t \times t$ kernel matrix $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = |\,\langle\,\Phi(\boldsymbol{x}_i)\,|\,\Phi(\boldsymbol{x}_j)\,\rangle\,|^2$, c.f. section VII. For all pairs of points $\boldsymbol{x}_i, \boldsymbol{x}_j \in T$ in the the training data, we sample the overlap between feature states to obtain the matrix entry in the kernel. This fidelity can be estimated from the output probability of the circuit depicted in Fig. 5.b. by sampling the output distribution with R shots and only taking the 0^n count. The frequency of the 0^n count is en estimator of the Kernel entry up to an error $\epsilon = \mathcal{O}(R^{-1/2})$. After the kernel matrix for the full training data has been constructed we use the conventional (classical) support vector machine classifier [1, 2]. The optimal hyperplane can be found by solving the dual quadratic program L_D for the variables $\boldsymbol{\alpha} = \{\alpha_i\}_{i=1\ldots t}$ described in section III A eqn. (10). Hence, to train, we maximize

$$L_D(\boldsymbol{\alpha}) = \sum_{i=1}^{t} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{t} y_i y_j \alpha_i \alpha_j K(\boldsymbol{x}_i, \boldsymbol{x}_j), \tag{1}$$

subject to $\sum_{i=1}^{t} \alpha_i y_i = 0$ and $\alpha_i \geq 0$. This problem is concave, and therefore efficiently solvable, whenever $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ is a positive definite matrix. Standard quadratic programming solvers can be used [3]. The solution to this problem

will be given by a nonnegative vector $\boldsymbol{\alpha}^* = (\alpha_1^*, \ldots, \alpha_t^*)$. Due to complementary slackness, c.f. eqn. (18) we expect that many of the α_i^* will be zero. Hence, there will be only subset of training samples that are needed to construct the optimal hyperplane. These samples are referred to as the support vectors.

The training phase consists of the following steps:

Algorithm 3 Quantum kernel estimation: the training phase

Input Labeled training samples $T = \{\boldsymbol{x} \in \Omega \subset \mathbb{R}^n\} \times \{y \in C\}$, quadratic program solver.
Parameters Number of measurement shots R.
Calibrate the quantum hardware to generate short depth circuits.
for $i = 1$ `to` t **do**
  for $j = 1$ `to` t **do**
    Set the counter $\mathrm{r}_{0^n} = 0$
    for $shot = 1$ `to` R **do**
      Run circuit from Fig. 5.b with parameters $\boldsymbol{x}_i, \boldsymbol{x}_j$.
      Measure outcome in Z-basis.
      if Measurement outcome is 0^n **then**
        Increase counter by one, setting $\mathrm{r}_{0^n} \to \mathrm{r}_{0^n} + 1$.
      end if
    end for
    Construct kernel estimate $\hat{K}(\boldsymbol{x}_i, \boldsymbol{x}_j) = \mathrm{r}_{0^n} R^{-1}$.
  end for
end for
Use quadratic program solver to optimize $\boldsymbol{\alpha}$ in L_D in eqn. (1) with kernel $K = \hat{K}$ and set T.
return the final parameters $\boldsymbol{\alpha}^*$ and value of the cost function L_D and kernel estimator $\hat{K}$.

In the classification phase, we want to assign a label to a new datum $\boldsymbol{s} \in S$ of the test set. For this, the inner products $K(\boldsymbol{x}_i, \boldsymbol{s})$ between all support vectors $\boldsymbol{x}_j \in T$ with $\alpha_i^* > 0$ and the new datum $\boldsymbol{s}$ have to be estimated on the quantum computer, c.f. Fig. 5.b. The new label $\tilde{m}(\boldsymbol{s})$ for the datum is assigned according to eqn. (19). Since all support vectors are known from the training phase and we have obtained access to the kernel $K(\boldsymbol{x}_i, \boldsymbol{s})$ from the quantum hardware, the label can be directly computed according to

$$\tilde{m}(\boldsymbol{s}) = \operatorname{sign}\left(\sum_{i=1}^{t} y_i \alpha_i^* K(\boldsymbol{x}_i, \boldsymbol{s}) + b\right).$$

Note that the bias b in $\tilde{m}(\boldsymbol{s})$ can calculated from the weights α_i^* by choosing any i with $\alpha_i^* > 0$ and solving $\sum_j y_j \alpha_j^* K(\boldsymbol{x}_j, \boldsymbol{x_i}) + b = y_i$ for b.

Algorithm 4 Quantum kernel estimation: the classification phase

Input An unlabeled sample from the test set $\boldsymbol{s} \in S$, optimal SVM parameters $\boldsymbol{\alpha}^*$, b and training data set T.
Parameters Number of measurement shots R
Calibrate the quantum hardware to generate short depth circuits.
for $i = 1$ `to` t `with` $\alpha_i^* > 0$ **do**
  Set the counter $\mathrm{r}_{0^n} = 0$.
  for $shot = 1$ `to` R **do**
    Run circuit from Fig. 5.b with parameters $\boldsymbol{x}_i, \boldsymbol{s}$
    Measure outcome in Z-basis
    if Measurement outcome is 0^n **then**
      Increase counter by one, setting $\mathrm{r}_{0^n} \to \mathrm{r}_{0^n} + 1$
    end if
  end for
  Construct kernel estimate $\hat{K}(\boldsymbol{x}_i, \boldsymbol{s}) = \mathrm{r}_{0^n} R^{-1}$.
end for
return $\operatorname{sign}\left(\sum_{i=1}^{t} y_i \alpha_i^* \hat{K}(\boldsymbol{x}_i, \boldsymbol{s}) + b\right)$.

III. THE RELATIONSHIP OF VARIATIONAL QUANTUM CLASSIFIERS TO SUPPORT VECTOR MACHINES

The references [1, 2] provide a detailed introduction to the construction of support vector machines for pattern recognition. Support vector machines are an important tool to construct classifiers for tasks in supervised learning. We will show that the variational circuit classifier bears many similarities to a classical non-linear support vector machine.

A. Support vector machines (SVMs)

First, let us briefly review the training task of classical, linear support vector machines for data where $C = \{+1, -1\}$, so that $(\boldsymbol{x}_i, y_i)_{i=1,\ldots,t}$ with $\boldsymbol{x}_i \in T \subset \mathbb{R}^d$, $y_i \in \{+1, -1\}$ that is linearly separable. Linear separability asks that the set of points can be split in two regions by a hyperplane $(\mathbf{w}, b)$, parametrized by a normal vector $\mathbf{w} \in \mathbb{R}^d$ and a bias $b \in \mathbb{R}$. The points $\boldsymbol{x} \in \mathbb{R}^d$ that lie directly on the hyperplane satisfy the equation

$$\mathbf{w} \circ \boldsymbol{x} + b = 0 \tag{2}$$

expressed in terms of the inner product $\circ$ for vectors in $\mathbb{R}^d$. The perpendicular distance of the hyperplane to the origin in $\mathbb{R}^n$ is given by $b\|\mathbf{w}\|^{-1}$. The data set $\{\boldsymbol{x}_i, y_i\}$ is linearly separable by margin $2||\mathbf{w}||^{-1}$ in $\mathbb{R}^d$ if there exists a vector $\mathbf{w}$ and a b, such that:

$$y_i\left(\mathbf{w} \circ \boldsymbol{x}_i + b\right) \geq 1. \quad \forall i = 1, \ldots, t \tag{3}$$

The classification function $\tilde{m}(\boldsymbol{x})$ that is constructed from such a hyperplane for any new data point $\boldsymbol{x} \in \mathbb{R}^n$ assigns the label according to which side of the hyperplane the new data-point lies by setting

$$\tilde{m}(\boldsymbol{x}) = \operatorname{sign}\left(\mathbf{w} \circ \boldsymbol{x} + b\right). \tag{4}$$

The task in constructing a linear support vector machine (SVM) in this scenario is the following. One is looking for a hyperplane that separates the data, with the largest possible distance between the two separated sets. The perpendicular distance between the plane and two points with different labels is called a margin and such points are referred to as 'support vectors'. This means that we want to maximize the margin by minimizing $||\mathbf{w}||$, or equivalently $||\mathbf{w}||^2$ subject to the constraints as given in eqn. (3), for all data points in the training set T. The corresponding cost function can be written as:

$$\text{minimize} \quad L_P = \frac{1}{2}\|\mathbf{w}\|^2 \tag{5}$$

$$\text{subject to:} \quad y_i\left(\mathbf{w} \circ \boldsymbol{x}_i + b\right) \geq 1. \quad \forall i = 1, \ldots, t \tag{6}$$

This can be expressed in terms of the Lagrangian

$$L(\mathbf{w}, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{t} \alpha_i y_i (\mathbf{w} \circ \boldsymbol{x}_i + b) + \sum_{i=1}^{t} \alpha_i, \tag{7}$$

where $\alpha_i \geq 0$ are Lagrange multipliers chosen to ensure the constraints are satisfied.

For non-separable datasets, it is possible to introduce non-negative slack variables $\{\xi_i\}_{i=1,\ldots,t} \in \mathbb{R}_0^+$ which can be used to soften the constraints for linear separability of $(\boldsymbol{x}_i, y_i)$ to

$$\begin{aligned} y_i\left(\mathbf{w} \circ \boldsymbol{x}_i + b\right) &\geq (1 - \xi_i), \\ \xi_i &\geq 0. \end{aligned} \tag{8}$$

These slack variables are then used to modify the objective function by $1/2\|\mathbf{w}\|^2 \to 1/2\|\mathbf{w}\|^2 + C(\sum_i \xi_i)^r + \sum_i \mu_i \xi$. When we choose $r \geq 1$ the optimization problem remains convex and a dual can be constructed. In particular, for $r = 1$, neither the ξ_i nor their Lagrange multipliers μ_i will appear in the dual Lagrangian.

The method can be generalized to the case when the decision function does depend non-linearly on the data by using a trick from [4] and introducing a high-dimensional, non-linear feature map. The data is mapped via a non-linear map

$$\Phi : \mathbb{R}^d \to \mathcal{H} \tag{9}$$

from a low dimensional space in to a high dimensional Hilbert-space $\mathcal{H}$. This space is commonly referred to as the feature space. If a suitable feature map has been chosen, it is then possible to apply the SVM classifier for the mapped data in $\mathcal{H}$, rather than in $\mathbb{R}^d$. This way it is possible to draw exceedingly complex decision surfaces that separate the data points based on their mapped features.

In light of the fact that the optimization problem now has to be solved in a much higher dimensional feature space with $d \ll \dim(\mathcal{H})$ for the vectors $\Phi(\boldsymbol{x})$ with $\boldsymbol{x} \in T$ it is very helpful to consider the dual of the original primal problem L_P in eqn. (5). The primal problem is a convex, quadratic programming problem, for which the Wolfe dual cost function L_D for the Lagrange multipliers can be readily derived by variation with respect to $\mathbf{w}$ and b. The dual optimization problem is expressed as an optimization of the Lagrange multipliers $\boldsymbol{\alpha} = \{\alpha_i\}_{i=1\ldots t}$

$$\text{maximize} \quad L_D = \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \Phi(\boldsymbol{x}_i) \circ \Phi(\boldsymbol{x}_j), \tag{10}$$

$$\text{subject to:} \quad \sum_i \alpha_i y_i = 0, \tag{11}$$

$$0 \leq \alpha_i \leq C, \quad \forall i = 1, \ldots, t. \tag{12}$$

The variables of the primal can be expressed in terms of the optimal dual variables $\boldsymbol{\alpha}^* = \{\alpha_i^*\}_{i=1\ldots t}$ as

$$\sum_i \alpha_i^* y_i \Phi(\boldsymbol{x}_i) = \mathbf{w} \tag{13}$$

and the bias b can be computed from the Karush-Kuhn-Tucker (KKT) conditions when the corresponding Lagrange multiplier α_i^* does not vanish. The optimal variables satisfy the KKT conditions and play an important role in the understanding of the SVM. They are given for the primal as

$$\partial_{w_\beta} L(\mathbf{w}, \boldsymbol{\alpha}) = w_\beta - \sum_i \alpha_i y_i \Phi_\beta(\boldsymbol{x}_i) = 0 \quad \text{for} \quad \beta = 1, \ldots, \dim(\mathcal{H}). \tag{14}$$

$$\partial_b L(\mathbf{w}, \boldsymbol{\alpha}) = -\sum_i \alpha_i y_i = 0 \tag{15}$$

$$\alpha_i \geq 0 \tag{16}$$

$$y_i \left(\Phi(\boldsymbol{x}_i) \circ \mathbf{w} + b\right) - 1 \geq 0 \tag{17}$$

$$\alpha_i \left(y_i(\mathbf{w} \circ \Phi(\boldsymbol{x}_i) + b) - 1\right) = 0. \tag{18}$$

Note that the condition eqn. (18) ensures that either the optimal $\alpha_i^* = 0$ or the corresponding constraint eqn. (18) is tight. This is a property referred to as complementary slackness, and indicates that only the vectors for which the constraint is tight give rise to non-zero $\alpha_i^* > 0$. These vectors are referred to as the support vectors and we will write N_S for their index set. The classifier in the dual picture is given by substituting $\mathbf{w}$ from eqn. (13) and b into the classifier eqn. (4). The bias b is obtained for any $i \in N_S$ from the equality in eqn. (17).

To understand why it is possible to solve the dual optimization problem in high dimensional space efficiently in the number of training samples t, it is important to note that it is in fact not necessary to construct the mapped data $\Phi(\boldsymbol{x}_i)$ in $\mathcal{H}$ explicitly. Both the training data, as well as the new data to be classified, enters only through inner products in both the optimization problem for training, c.f. eqn. (10) and in the classifier, eqn. (4). Hence, we can construct the SVM for arbitrarily high dimensional feature maps $(\Phi, \mathcal{H})$, if we can efficiently evaluate the inner products $\Phi(\boldsymbol{x}_i) \circ \Phi(\boldsymbol{x}_j)$ and $\Phi(\boldsymbol{x}_i) \circ \Phi(\boldsymbol{s})$, for $\boldsymbol{x}_i \in T$ and $\boldsymbol{s} \in S$. In particular, if we can evaluate the kernel $K(\boldsymbol{x}, \boldsymbol{y}) = \Phi(\boldsymbol{x}) \circ \Phi(\boldsymbol{y})$ [1, 4], we can construct a classifier by setting

$$\tilde{m}(\boldsymbol{s}) = \text{sign}\left(\sum_{i \in N_S} \alpha_i^* y_i K(\boldsymbol{x}_i, \boldsymbol{s}) \; + b\right). \tag{19}$$

Here we only need to sum over all support vectors $i \in N_S$ for which $\alpha_i^* > 0$. Moreover, one can replace the inner product in the optimization problem eqn. (10) by the kernel.

Examples of kernels frequently considered in the classical literature are for instance the polynomial kernel $K(\boldsymbol{x}, \boldsymbol{y}) = (\boldsymbol{x} \circ \boldsymbol{y} + 1)^m$ or the infinite dimensional Gaussian kernel $K(\boldsymbol{x}, \boldsymbol{y}) = \exp(-1/2\|\boldsymbol{x} - \boldsymbol{y}\|^2)$. If the feature map is sufficiently powerful, increasingly complex distributions can be classified.

In this paper, the feature map is a classical to quantum mapping by a tunable quantum circuit family, that maps $\Phi : \mathbb{R}^d \to \mathcal{S}(\mathcal{H}_2^{\otimes n})$ in to the state space (space of density matrices) of n qubits with $\dim\left(\mathcal{S}(\mathcal{H}_2^{\otimes n})\right) = 4^n$. The example of the Gaussian kernel indicates, that the sheer dimension of the Hilbert space available on a quantum computer by itself does not provide an advantage, since even classically infinite dimensional spaces are available (e.g. the Gaussian kernel). However, this procedure provides a path towards a quantum advantage as we may construct states in feature space with classically hard-to-estimate overlaps.

B. Variational circuit classifiers

Let us now turn to the case of binary classification based on variational quantum circuits. Recall that in our setting, we first take the data $\boldsymbol{x} \in \mathbb{R}^d$ and map it to a quantum state $|\,\Phi(\boldsymbol{x})\rangle\langle\Phi(\boldsymbol{x})\,| \in \mathcal{S}(\mathcal{H}_2^{\otimes n})$ on n-qubits, c.f. eqn. (27). Then we apply a variational circuit $W(\boldsymbol{\theta})$ to the initial state that depends on some variational parameters $\boldsymbol{\theta}$, c.f. eqn. (38). Lastly, for a binary classification task, we measure the resulting state in the canonical Z-basis and assign the resulting bit-string $z \in \{0,1\}^n$ to a label based on a predetermined boolean function $f : \{0,1\}^n \to \{+1,-1\}$. Hence the probability of measuring either label $y \in \{+1,-1\}$ is given by:

$$p_y = \frac{1}{2}\left(1 + y\langle\Phi(\boldsymbol{x})\,|W^\dagger(\boldsymbol{\theta})\;\mathbf{f}\;W(\boldsymbol{\theta})|\,\Phi(\boldsymbol{x})\rangle\right), \tag{20}$$

where we have defined the diagonal operator

$$\mathbf{f} = \sum_{z\in\{0,1\}^n} f(z)|\,z\rangle\langle z\,|. \tag{21}$$

In classification tasks we assign c.f. eqn. (41), the label with the highest empirical weight of the distribution p_y. We ask whether the outcome $+1$ is more likely than -1, or vice versa. That is, we ask, whether $p_{+1} > p_{-1} - b$ or whether the converse is true. This of course depends on the sign of the expectation value $\langle\Phi(\boldsymbol{x})\,|W^\dagger(\boldsymbol{\theta})\;\mathbf{f}\;W(\boldsymbol{\theta})|\,\Phi(\boldsymbol{x})\rangle$ for the data point $\boldsymbol{x}$.

To understand how this relates to the SVM in greater detail, we need to choose an orthonormal operator basis, such as for example the Pauli group

$$\mathcal{P}_n = \langle X_i, Y_i, Z_i\rangle_{i=1,\ldots,n}\,. \tag{22}$$

Note that when fixing the phase to $+1$ every element $P_\beta \in \mathcal{P}_n$, with $\beta = 1,\ldots,4^n$ of the Pauli-group is an orthogonal reflection $P_\beta^2 = \mathbb{1}$. Furthermore, Pauli matrices are mutually orthogonal in terms of the trace inner product

$$\mathrm{tr}\left[P_\alpha P_\beta\right] = \delta_{\alpha,\beta}2^n. \tag{23}$$

This means that both the measurement operator $\mathbf{w} = W^\dagger(\boldsymbol{\theta})\,\mathbf{f}\,W(\boldsymbol{\theta})$ in the W-rotated frame as well as the state $\Phi(\boldsymbol{x}) = |\,\Phi(\boldsymbol{x})\rangle\langle\Phi(\boldsymbol{x})\,|$ can be expanded in terms of the operator basis with only real coefficients as

$$\begin{aligned}\mathbf{w} &= \frac{1}{2^n}\sum_\beta w_\beta(\theta)P_\beta \quad \text{with} \quad w_\beta(\boldsymbol{\theta}) = \mathrm{tr}\left[W^\dagger(\boldsymbol{\theta})\;\mathbf{f}\;W(\boldsymbol{\theta})P_\beta\right]\\ \Phi(\boldsymbol{x}) &= \frac{1}{2^n}\sum_\beta \Phi_\beta(\boldsymbol{x})P_\beta \quad \text{with} \quad \Phi_\beta(\boldsymbol{x}) = \mathrm{tr}\left[|\,\Phi(\boldsymbol{x})\rangle\langle\Phi(\boldsymbol{x})\,|P_\beta\right].\end{aligned} \tag{24}$$

Note, that the values $\mathbf{w}_\beta(\boldsymbol{\theta})$ as well as $\Phi_\beta(\boldsymbol{x})$ are constrained due to the fact that they originate from a rotated projector and from a pure state. Since $\mathbf{f}^2 = \mathbb{1}$, we have that $\mathrm{tr}\left[\left(W^\dagger(\boldsymbol{\theta})\;\mathbf{f}\;W(\boldsymbol{\theta})\right)^2\right] = 2^n$. Furthermore, the projector squares to itself so that $\mathrm{tr}\left[|\,\Phi(\boldsymbol{x})\rangle\langle\Phi(\boldsymbol{x})\,|^2\right] = 1$. In particular, this means that the norms of both vectors satisfy $\sum_\beta \Phi_\beta^2(\boldsymbol{x}) = 2^n$ as well as $\sum_\beta w_\beta^2(\theta) = 4^n$.

Since the expectation value of the measured observable is $\langle\Phi(\boldsymbol{x})\,|W^\dagger(\boldsymbol{\theta})\mathbf{f}W(\boldsymbol{\theta})|\,\Phi(\boldsymbol{x})\rangle = \mathrm{tr}\left[\mathbf{w}\;\Phi(\boldsymbol{x})\right]$, we can expressed this Hilbert-Schmidt inner-product explicitly in the Pauli basis:

$$\langle\Phi(\boldsymbol{x})\,|W^\dagger(\boldsymbol{\theta})\;\mathbf{f}\;W(\boldsymbol{\theta})|\,\Phi(\boldsymbol{x})\rangle = \frac{1}{2^n}\sum_\beta w_\beta(\boldsymbol{\theta})\Phi_\beta(\boldsymbol{x}). \tag{25}$$

Observe that $\mathbf{f}$ only has eigenvalues $\{+1,-1\}$, and we have that $\langle\Phi(\boldsymbol{x})\,|W^\dagger(\boldsymbol{\theta})\,\mathbf{f}\,W(\boldsymbol{\theta})|\,\Phi(\boldsymbol{x})\rangle \in [-1,+1]$. Let us now consider a decision rule, where we assign the label $y \in \{+1,-1\}$ over the label $-y$ with some fixed bias $b \in [-1,+1]$. In that case we demand that $p_y > p_{-y} - yb$. If we substitute eqn. (20) and use the expansion in eqn. (25) we have that the corresponding label is given by the decision function $y = \tilde{m}(\boldsymbol{x})$, where

$$\tilde{m}(\boldsymbol{x}) = \operatorname{sign}\left(\frac{1}{2^n}\sum_\beta w_\beta(\theta)\Phi_\beta(\boldsymbol{x}) + b\right). \tag{26}$$

This expression is identical to the linear threshold function used in conventional SVM classifier, c.f. eqn. (4), after the feature map has been applied. However, in the experiment we only have access to the probabilities p_y through sample estimates. Furthermore, the $w_\beta(\boldsymbol{\theta})$ are constrained to stem from the observable $\mathbf{f}$ measured in the rotated frame. If this constraint is dropped, and we are able to optimize over all possible hyperplanes in feature space, the classifier can be improved.

Furthermore, this decomposition indicates that the correct feature space, where a linearly separating hyperplane is constructed, is in fact the quantum state space of density matrices, and not the Hilbert space $\mathcal{H}_2^n$ itself. The physical states in $\mathcal{H}_2^n$ are only defined up to a global phase $|\,\psi\rangle \sim e^{i\eta}|\,\psi\rangle$. The equivalence of states up to a global phase would make it impossible to find a separating hyperplane in this space, since both $|\,\psi\rangle$ and $-|\,\psi\rangle$ give rise to the same physical state but can lie on either side of a separating plane determined by an inner-product $\langle\,\mathbf{w}\,|\,\psi\,\rangle$.

Hence, it is evident that a quantum circuit can only generate linear decision surfaces since it acts only as a unitary on the input state. The complex transformation, i.e. the non-linear mapping to the feature space, has to occur when the data itself is mapped to the quantum computer. Therefore, if we want to improve the classifier over what can be accomplished classically the improvement has to occur at this step. If the classical data is mapped to a quantum state in such a fashion that the inner-products of the states can be evaluated efficiently classically no quantum advantage can be achieved. The classical optimization problem in the SVM is a concave problem in $|T|^2$ variables, which means that it has a unique solution that can be found efficiently. The use of a conventional SVM presents an advantage over using a variational circuit in that it will not get stuck in local minima and can optimize over all hyperplanes in the state space.

IV. ENCODING OF THE DATA USING A SUITABLE FEATURE MAP

In the quantum setting, the feature map is an injective encoding of classical information $\boldsymbol{x} \in \mathbb{R}^d$ into a quantum state $|\,\Phi\rangle\langle\Phi\,|$ on an n-qubit register. Here $\mathcal{H}_2 = \mathbb{C}^2$ is a single qubit Hilbert space, and $\mathcal{S}\left(\mathcal{H}_2^{\otimes n}\right)$ denotes the cone of positive semidefinite density matrices $\rho \geq 0$ with unit trace $\operatorname{tr}[\rho] = 1$. This cone is a subset of the 4^n dimensional Hilbert space of $\mathcal{M}_{2^n\times 2^n}(\mathbb{C})$ of complex matrices when fitted with the inner product $\operatorname{tr}\left[A^\dagger B\right]$ for $A, B \in \mathcal{M}_{2^n\times 2^n}(\mathbb{C})$. The feature map acts as

$$\Phi : \Omega \subset \mathbb{R}^d \to \mathcal{S}\left(\mathcal{H}_2^{\otimes n}\right), \qquad \Phi : \boldsymbol{x} \mapsto |\,\Phi(\boldsymbol{x})\rangle\langle\Phi(\boldsymbol{x})\,|. \tag{27}$$

The action of the map can be understood by a unitary circuit family denoted by $\mathcal{U}_{\Phi(\boldsymbol{x})}$ that is applied to some reference state, e.g. $|\,0\rangle^n$. The resulting state is given by $|\,\Phi(\boldsymbol{x})\rangle = \mathcal{U}_{\Phi(\boldsymbol{x})}|\,0\rangle^n$. The state in the feature space should depend non-linearly on the data. Let us discuss proposals for possible feature maps.

Product state feature maps There are many choices for the feature map Φ. Let us first discuss what would happen if we were to choose a feature map that corresponds to a product input state. We assume a feature map, comprised of single qubit rotations $U(\varphi) \in \mathrm{SU}(2)$ on every qubit of the quantum circuit. The angles for every qubit can be chosen as a non-linear function $\varphi : \boldsymbol{x} \to (0, 2\pi]^2 \times [0,\pi]$ into the space of Euler angles for the individual qubits, so that the full feature map can be implemented as:

$$\boldsymbol{x} \mapsto |\,\phi_i(\boldsymbol{x})\rangle = U(\varphi_i(\boldsymbol{x}))|\,0\rangle, \quad \text{for an individual qubit, so that} \tag{28}$$

$$\Phi : \boldsymbol{x} \mapsto |\,\Phi(\boldsymbol{x})\rangle\langle\Phi(\boldsymbol{x})\,| = \bigotimes_{i=1}^{n} |\,\phi_i(x)\rangle\langle\phi_i(x)\,| \quad \text{for the full qubit state.} \tag{29}$$

One example for such an implementation is the unitary implementation of the feature map used in the context of the classical classifiers by Stoudenmire and Schwab [5] based on tensor networks. There each qubit encodes a single

component x_i of $\boldsymbol{x} \in [0,1]^n$ so that n qubits are used. The resulting state that is prepared is

$$\bigotimes_{i=1}^{n} |\,\phi_i(x)\rangle\langle\phi_i(x)\,| = \frac{1}{2^n}\bigotimes_{j=1}^{n}\left(\sum_{\alpha_j}\Phi_j^{\alpha_j}\left(\theta_j(\boldsymbol{x})\right)P_{\alpha_j}\right), \tag{30}$$

when expanded in terms of the Pauli-matrix basis where $\Phi_i^{\alpha}(\theta_i(\boldsymbol{x})) = \langle\phi_i(x)\,|P_{\alpha_i}|\,\phi_i(x)\rangle$ for all $i = 1,\ldots n$. and $P_{\alpha_i} \in \{\mathbb{1}, X_i, Z_i, Y_i\}$. The corresponding decision function can be constructed as in eqn. (19), with the kernel $K(\boldsymbol{x},\boldsymbol{y}) = \prod_{i=1}^{n} |\,\langle\,\phi_i(\boldsymbol{x})\,|\,\phi_i(\boldsymbol{y})\,\rangle\,|^2$. These can be evaluated with resources scaling linearly in the number of qubits, so that no quantum advantage can be expected in this setting.

A. Non-trivial feature map with entanglement

There are many choices of feature maps, that do not suffer from the malaise of the aforementioned product state feature maps. To obtain a quantum advantage we would like these maps to give rise to a kernel $K(\boldsymbol{x},\boldsymbol{y}) = |\,\langle\,\Phi(\boldsymbol{x})\,|\,\Phi(\boldsymbol{y})\,\rangle\,|^2$ that is computationally hard to estimate up to an additive polynomially small error by classical means. Otherwise, the map is immediately amenable to classical analysis and we are guaranteed to have lost any conceivable quantum advantage.

Let us therefore turn to a family of feature maps, c.f. Fig 2 for which we conjecture that it is hard to estimate the overlap $|\,\langle\,\Phi(\boldsymbol{x})\,|\,\Phi(\boldsymbol{y})\,\rangle\,|^2$ on a classical computer. We define the family of feature map circuit as follows

$$|\,\Phi(\boldsymbol{x})\rangle \;=\; U_{\Phi(\boldsymbol{x})}\,H^{\otimes n}\,U_{\Phi(\boldsymbol{x})}H^{\otimes n}|\,0\rangle^{\otimes n} \quad \text{where,} \quad U_{\Phi(\boldsymbol{x})} \;=\; \exp\left(i\sum_{S\subseteq[n]}\phi_S(\boldsymbol{x})\prod_{i\in S}Z_i\right). \tag{31}$$

where the 2^n possible coefficients $\phi_S(\boldsymbol{x}) \in \mathbb{R}$ are now non-linear functions of the input data $\boldsymbol{x} \in \mathbb{R}^n$. It is convenient to use maps with low-degree expansions, i.e. $|S| \leq d$. Any such map can be efficiently implemented. In the experiments reported in this paper we have restricted to $d = 2$. So we only consider Ising type interactions in the unitaries $U_{\Phi(\boldsymbol{x})}$. It is convenient choose these interactions as the ones that are present in the actual connectivity graph of the superconducting chip $G = (E, V)$. This ensures that the feature map can be generated from a short depth circuit. The resulting unitary can then be generated from one- and two- qubit gates of the form

$$U_{\phi_{\{l,m\}}(\boldsymbol{x})} = \exp\left(i\phi_{\{k,l\}}(\boldsymbol{x})Z_kZ_l\right) \quad \text{and} \quad U_{\phi_{\{k\}}(\boldsymbol{x})} = \exp\left(i\phi_{\{k\}}(\boldsymbol{x})Z_k\right), \tag{32}$$

which leaves $|V| + |E|$, real parameters to encode the data. In particular, we know that we have at least $|V| = n$ real numbers to encode the data. Furthermore, depending on the connectivity of all possible interactions, we have $|E| \leq n(n-1)/2$ further parameters that can be used to encode more data or nonlinear relations of the initial data points.

This feature map encodes both the actual function $\Phi_{\boldsymbol{x}}(z)$ of the diagonal phases, as well as the corresponding Fourier-Walsh transform $\hat{\Phi}_{\boldsymbol{x}}(p)$ at $z, p \in \{0,1\}^n$

$$\Phi_{\boldsymbol{x}}(z) = \exp\left(i\sum_{S\subseteq[n]}\phi_S(\boldsymbol{x})\prod_{i\in S}(-1)^{z_i}\right) \quad \text{and} \quad \hat{\Phi}_{\boldsymbol{x}}(p) = \frac{1}{2^n}\sum_{z\in\{0,1\}^n}\Phi_{\boldsymbol{x}}(z)(-1)^{p\circ z}, \tag{33}$$

for every basis element respectively. The resulting state, after the datum $\boldsymbol{x} \in \mathbb{R}^d$ has been mapped to the feature space, is given by

$$|\,\Phi(\boldsymbol{x})\rangle = \sum_{p\in\{0,1\}^N}\Phi_{\boldsymbol{x}}(p)\hat{\Phi}_{\boldsymbol{x}}(p)|\,p\rangle. \tag{34}$$

We conjecture that it is hard to estimate this kernel up to an additive polynomially small error by classical means. The intuition for the conjecture stems from a connection of the feature map to a particular circuit family for the hidden shift problem for boolean functions [6]. The feature map is similar to a quantum algorithm introduced by Rötteler for estimating the hidden shift in boolean bent functions, c.f. Ref. [7], algorithm $\mathcal{A}_1$ in Theorem 6. The circuit $\mathcal{U}_\Phi$ we consider is indeed very similar to the one discussed in [7]. Let us make a minor modification, and ask the

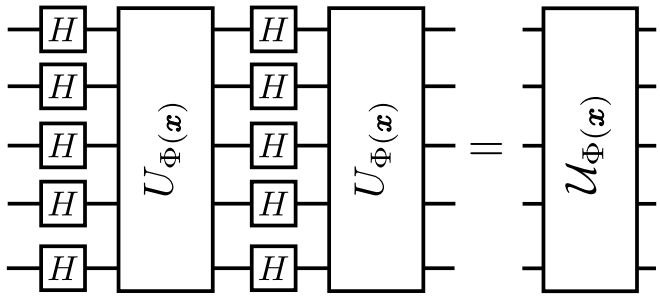

FIG. 2. A circuit representation of the feature map family we consider here. We first apply a series of Hadamard gates before applying the diagonal phase gate component. Then we apply a second layer of Hadamard gates, followed by the same diagonal phase gate. This encodes both the actual function value of the phase $\Phi_{\boldsymbol{x}}(z)$ as well as the value of the $\mathbb{Z}_2^N$ Fourier transform $\hat{\Phi}_{\boldsymbol{x}}(S)$ for every basis element.

two diagonal layers in $\tilde{\mathcal{U}}_\Phi = U_{\Phi^2} H^{\otimes n} U_{\Phi^1} H^{\otimes n}$ to differ. We choose the phase gate layers so that U_{Φ^1} and U_{Φ^2} encode a shifted bent-function and the function's dual, then the circuit $\mathcal{A}_1$ is given by $H^{\otimes n}\tilde{\mathcal{U}}_\Phi$. If the interaction graph G is bi-partite, it is possible to encode Maiorana-McFarland bent-functions for $d = 2$ by choosing the corresponding $\phi_{\{k,l\}}(\boldsymbol{x}), \phi_k(\boldsymbol{x})$ to be either π or 0. In [7], the diagonal layer in $\mathcal{A}_1$, are queries to an oracle encoding the the shifted bent-function and it's dual. It can be shown that with respect to this oracle there is an exponential separation in query complexity over classical query algorithms.

The final circuit we implement to estimate the overlap $|\langle \Phi(\boldsymbol{x}) | \Phi(\boldsymbol{y}) \rangle|^2$, c.f. Fig 5 b) is still larger. This circuit has four layers of Hadamard gates and 3 diagonal layers. A circuit that has this kind of structure is related to a problem introduced by Aaronson and Ambainis [8]. The problem is referred to k-fold Forrelation, c.f. [8], section 1.1.3. where given k total Boolean functions $f_i : \{0,1\} \to \{+1,-1\}$, $i = 1 \dots k$ one has to decide, whether the sum

$$F_{f_1,f_2,\dots,f_k} = \frac{1}{2^{(k+1)n/2}} \sum_{x_1,\dots x_k \in \{0,1\}^n} f_1(x_1)(-1)^{x_1 \circ x_2} f_2(x_2)(-1)^{x_2 \circ x_3} \dots (-1)^{x_{k-1} \circ x_k} f_k(x_k), \tag{35}$$

satisfies $|F_{f_1,f_2,\dots,f_k}| \leq 100^{-1}$ or whether $F_{f_1,f_2,\dots,f_k} \geq 3/5$, promised that one of these holds. The authors show, that any classical, randomized algorithm that has only black box access to the Boolean functions must make $\mathcal{O}(2^{n/2} n^{-1})$ queries to these black box functions to solve the problem. However, a quantum computer is able to solve this problem with k queries (in fact only $\lceil k/2 \rceil$ queries) to a phase oracle that encodes the Boolean function through $U_{f_i} |x\rangle = f_i(x) |x\rangle$ with bounded probability. The algorithm starts by preparing the initial state $|0\rangle^{\otimes n}$ and applies an alternating sequence of Hadamard gates $H^{\otimes n}$ and queries to the oracles U_{f_i} like

$$H^{\otimes n} \to U_{f_1} \to H^{\otimes n} \to U_{f_2} \to H^{\otimes n} \to \dots \to U_{f_k} \to H^{\otimes n}. \tag{36}$$

Finally if we measure all n - qubits in the Z - basis, we observe that probability to obtain the outcome $|0\rangle^{\otimes n}$ is exactly $|F_{f_1,f_2,\dots,f_k}|^2$. So if we compare k-fold Forrelation with $k = 3$ to the estimation of the quantum Kernel, $|\langle \Phi(\boldsymbol{x}) | \Phi(\boldsymbol{y}) \rangle|^2 = |\langle 0|^{\otimes n} H^{\otimes n} U^\dagger_{\Phi(\boldsymbol{y})} H^{\otimes n} U^\dagger_{\Phi(\boldsymbol{y})} U_{\Phi(\boldsymbol{x})} H^{\otimes n} U_{\Phi(\boldsymbol{x})} H^{\otimes n} |0\rangle^{\otimes n}$, we observe that the implementation of the phase gates $U_{\Phi(\boldsymbol{x})}$, $U^\dagger_{\Phi(\boldsymbol{y})} U_{\Phi(\boldsymbol{x})}$ and $U^\dagger_{\Phi(\boldsymbol{y})}$ assume the roles of the oracles U_{f_1} U_{f_2} and U_{f_3} queried in 3-fold Forrelation respectively. Hence, the kernel estimation we perform here is directly related to the Forrelation problem by Aaronson and Ambainis.

If the conjecture, that estimating the kernel entries is hard could be proven, it would establish a valuable step in rigorously establishing a quantum advantage on near-term devices. If, however, it ultimately turns out that this family can also be evaluated classically, we would need to improve the complexity of this circuit family. This could be achieved by increasing the number of layers and / or the interactions present in the diagonal unitary.

A good example of a circuit family that entails a hardness result, yet does not provide an advantage in our setting is the following: One could consider the case, where only a single layer of Hadamard gates and a single diagonal unitary is applied. Such a feature map is directly related to the circuit family introduced in [9]. Indeed the resulting kernel $K(\boldsymbol{x}, \boldsymbol{y}) = |\langle 0^n | H^{\otimes n} U_{\Phi(\boldsymbol{y})} U_{\Phi(\boldsymbol{x})} H^{\otimes n} | 0^n \rangle|^2$ corresponds to the output probability of the considered

IQP circuits. In fact it is known that if general real values $\phi_S(\boldsymbol{x})$ are allowed that $d = 2$ suffices to encode $\#P$-hard problems in the output probability [10]. This hardness result, only applies to the case where the output probability can be approximated up to a multiplicative error. A noisy quantum computer, however, is also not able to provide a multiplicative error estimate to this quantity. Nevertheless, Bremner et al. show, contingent on additional complexity theoretic conjectures, that it is hard for a classical sampler to produce samples from the output distribution of the IQP circuit. Does this result imply some form of hardness for the feature map version of this circuit? The answer is unfortunately no. The kernel can be estimated up to an additive error of ϵ by drawing $R = \epsilon^{-2}$ samples from the uniform distribution over n classical bits and averaging $\tilde{\Phi}_{\boldsymbol{x}-\boldsymbol{y}}(z) = \Phi_{\boldsymbol{x}}(z)\overline{\Phi}_{\boldsymbol{y}}(z)$, c.f. eqn. (33)

$$|\langle 0^n | H^{\otimes n} U_{\Phi(\boldsymbol{y})} U_{\Phi(\boldsymbol{x})} H^{\otimes n} | 0^n \rangle|^2 = \left| \frac{1}{2^n} \sum_{z \in \{0,1\}^n} \tilde{\Phi}_{\boldsymbol{x}-\boldsymbol{y}}(z) \right|^2 . \tag{37}$$

Since we have that the variance of the random variable is bounded by 1, i.e. $|\Phi_{\boldsymbol{x}}(z)|^2 = 1$, we get an additive error that scales as $\mathcal{O}(\epsilon)$. This means for a single layer, the kernel can be estimated classically.

V. QUANTUM VARIATIONAL CLASSIFICATION

Following the structure of the feature map circuit, we construct the classifier part of the variational algorithm by appending layers of single-qubit unitaries and entangling gates Fig. 3.a. Each subsequent layer, or depth, contains an additional set of entanglers across all the qubits used for the algorithm. We use a coherently controllable quantum mechanical system, such as for example the superconducting chip with n transmon qubits to prepare a short depth quantum circuit $W(\boldsymbol{\theta})$. In the experiment here, comprising $n = 2$ qubits, one controlled-phase gate is added per depth. The single-qubit unitaries used in the classifier are limited to Y and Z rotations to simplify the number of parameters to be handled by the classical optimizer. Our use of controlled-phase, rather than CNOT, gates for the entanglers is justified by our aim at increased generality in our software. Using controlled-phase gates does not require to particularize this part of the algorithm for different systems topologies. A specific entangling map for a given device can then be used by our compiler to translate each controlled-phase gate into the CNOTs available in our system.

The general circuit is comprised of the following sequence of single qubit and multi-qubit gates:

$$W(\boldsymbol{\theta}) = U^{(l)}_{\text{loc}}(\theta_l)\, U_{\text{ent}} \ldots U^{(2)}_{\text{loc}}(\theta_2)\, U_{\text{ent}}\, U^{(1)}_{\text{loc}}(\theta_1). \tag{38}$$

We apply a circuit of l repeated entanglers as depicted in Fig 3.b and interleave them with layers comprised of local single qubit rotations:

$$U^{(t)}_{\text{loc}}(\theta_t) = \otimes_{m=1}^{n} U(\theta_{m,t}) \quad \text{and} \quad U(\theta_{m,t}) = e^{i\frac{1}{2}\theta^z_{m,t} Z_m} e^{i\frac{1}{2}\theta^y_{m,t} Y_m}, \tag{39}$$

parametrized by $\theta_t \in \mathbb{R}^{2n}$ and $\theta_{i,t} \in \mathbb{R}^2$. In principle, there exist multiple choices for the entangling unitaries U_{ent} [11, 12]. For the feature map that we consider, however, we use the entangler that is comprised of products of control phase gates $\mathsf{CZ}(i,j)$ between qubits i and j. The entangling interactions follow the interaction graph that $G = (E, V)$ that was used to generate the feature map in eqn. (32).

$$U_{\text{ent}} = \prod_{(i,j)\in E} \mathsf{CZ}(i,j), \tag{40}$$

This short-depth circuit can generate any unitary if sufficiently many layers d are applied. The circuit Fig. 3.a can be understood as a bang-bang controlled evolution of an Ising model Hamiltonian $H_0 = \sum_{(ij)\in E} J_{ij} Z_i Z_j$,c.f. Fig. 3.b, interspersed with single qubit control pulses in $SU(2)$ on every qubit. It is known that this set of drift steps together with all single control pulses are universal, so we have that any state can be prepared this way with sufficient circuit depth [13]. For a general unitary gate sequence the entangling unitary has to be effectively generated from cross resonance gates, by applying single qubit control pulses.

VI. CHOOSING THE COST-FUNCTION FOR THE CIRCUIT OPTIMIZATION

The central goal of the quantum variational classifier is to find an optimal classifying circuit $W(\boldsymbol{\theta})$, c..f. eqn. (38) that separates the data sets with different labels. Since we can re-run the same classifying multiple times (R shots),

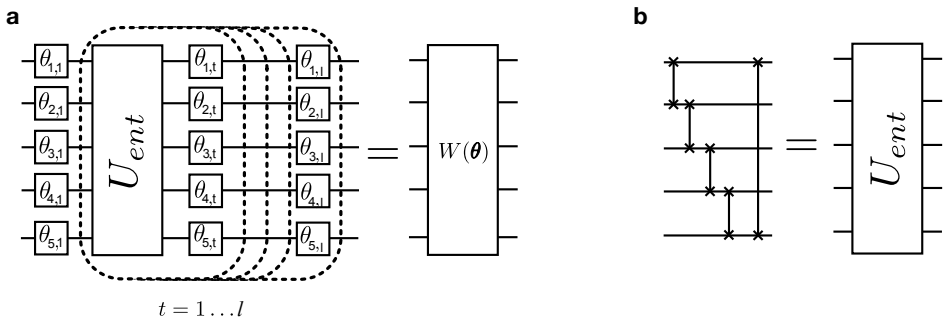

FIG. 3. (a) Circuit representation of short depth quantum circuit to define the separating hyperplane. The single qubit rotations $U(\theta_{i,t}) \in \mathrm{SU}(2)$ are depicted by single line boxes parametrized by the angles θ_i, while the entangling operation U_{ent} is determined by the interaction graph of the superconducting chip. (b) Depiction of entangling gate as a product of $\mathsf{CZ}_{i,i+1}$ for $i = 1, \dots, 5$ gates following the interaction graph of a circle $G = C_5$.

we may consider a 'winner takes all' scenario, where we assign the label according to the outcome with the largest probability estimate. We choose a cost function, so that the optimization procedure minimizes the probability of assigning the wrong label after having constructed the distribution after R shots.

There are various ways of performing a multi-label classification. We only need to modify the final measurement M, to correspond to multiple partitions. This can be achieved by multiple strategies. For example one could choose to measure again in the computational basis, i.e. the basis in which Pauli Z are diagonal and then constructing classical labels form the measured samples, such as a labeling the outcome $z \in \{0,1\}^n$ according to a function $f : \{0,1\}^n \to \{1, \dots, c\}$. The resulting $\{M_y\}_{y=1,\dots,c}$ is therefore diagonal in the computational basis. Alternatively one could construct a commuting measurement akin to the syndrome check measurement for quantum stabilizers. For this approach we choose a set $\{g_i\}_{i=1\dots\lceil\log_2(c)\rceil}$ of Pauli matrices $g_i \in \mathcal{P}_N$ that are commuting $[g_i, g_j] = 0$. The resulting measurement that would need to be performed is similar to that of an error correcting scheme. The measurement operators are given by $M_y = 2^{-1}\left(1 - \prod_{i=1}^{\lceil\log_2(c)\rceil} g_i^{y^i}\right)$. Here y^i denotes the i'th bit in the binary representation of y. In either case, the decision rule that assigns the labels can be written as

$$\tilde{m}_{|T}(\boldsymbol{x}) = \arg\max_{c'} \langle \Phi(\boldsymbol{x}) \left| W(\boldsymbol{\theta})^\dagger M_{c'} W(\boldsymbol{\theta}) \right| \Phi(\boldsymbol{x}) \rangle \,. \tag{41}$$

This corresponds to taking R shots in order to estimate the largest outcome probability from the outcome statistics of the measurement M_y for $y = 1, \dots, c$. Labelling T_c the subset of samples T labelled with c, the avarage expected misclassification rate is given by:

$$P_{\mathrm{err}} = \frac{1}{|T|}\left(\sum_c \sum_{\boldsymbol{s} \in T_c} \Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq c | \boldsymbol{s} \in T_c\right)\right). \tag{42}$$

The error probability of misclassifying an individual datum is written as $\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq c | \boldsymbol{s} \in T_c\right)$. This error probability is now used to define the empirical risk function $R_{\mathrm{emp}}(\boldsymbol{\theta}) = P_{\mathrm{err}}$. We discuss of how to find suitable ways of evaluating the cost function for this classification scheme.

A. Binary label classification

Assume the programmer classifies labels $y \in \{-1, 1\}$ by taking R shots for a single datapoint. She obtains an empirical estimates of probability of the datum being labeled by a label y

$$\hat{p}_y = \frac{r_y}{R} \,.$$

After $R = r_y + r_{-y}$ shots and a prior bias b, she misclassifies into a label y if

$$\hat{p}_y < \hat{p}_{-y} + yb \to r_y < r_{-y} + ybR \to r_y < \left\lceil \frac{1 - yb}{2} R \right\rceil$$

The probability of her misclassifying a y sample according to the argmax rule is hence estimated by

$$\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq y | \boldsymbol{s} \in T_y\right) = \Pr\left(r_y < r_{-y} - yb\right) = \sum_{j=0}^{\lceil \frac{1-yb}{2} R \rceil} \binom{R}{j} p_y^j (1-p_y)^{R-j} \,.$$

Assuming large R, computing this exactly may be difficult. Setting $Rp_y = a, Rp_y(1-p_y) = \beta^2$ and $\lceil \left(\frac{1+yb}{2}\right) R \rceil = \gamma$, we can approximate the binomial CDF as an error function:

$$\begin{aligned}
\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq y | \boldsymbol{s} \in T_y\right) &= \sum_{j=0}^{\lceil (\frac{1-yb}{2}) R \rceil} \binom{R}{j} p_c^{R-j} (1-p_c)^j \approx \int_{-\infty}^{\gamma} dx \frac{1}{\sqrt{2\pi}\beta} \exp\left(-\frac{1}{2}\left(\frac{x-a}{\beta}\right)^2\right) \\
&= \frac{1}{\sqrt{\pi}} \int_{-\infty}^{\frac{\gamma-a}{\sqrt{2}\beta}} dz\, e^{-z^2} = \frac{1}{2}\mathrm{erf}\left(\frac{\gamma-a}{\sqrt{2}\beta}\right) + \frac{1}{2} \\
&= \frac{1}{2}\mathrm{erf}\left(\sqrt{R}\frac{\frac{1-yb}{2} - p_y}{\sqrt{2(1-p_y)p_y}}\right) + \frac{1}{2} \,.
\end{aligned}$$

See Fig. 4 a. The error function can be consequently approximated with a sigmoid

$$\mathrm{sig}(x) = \frac{1}{1+\exp(-x)} \approx \frac{1}{2}\left(\mathrm{erf}(x) + 1\right),$$

which gives

$$\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq y | \boldsymbol{s} \in T_y\right) \approx \mathrm{sig}\left(\sqrt{R}\frac{\frac{1-yb}{2} - p_y}{\sqrt{2(1-p_y)p_y}}\right) . \tag{43}$$

as an estimate for misclassifying a sample $\boldsymbol{s}$. The cost function to optimize is then given by using this in eqn. (42).

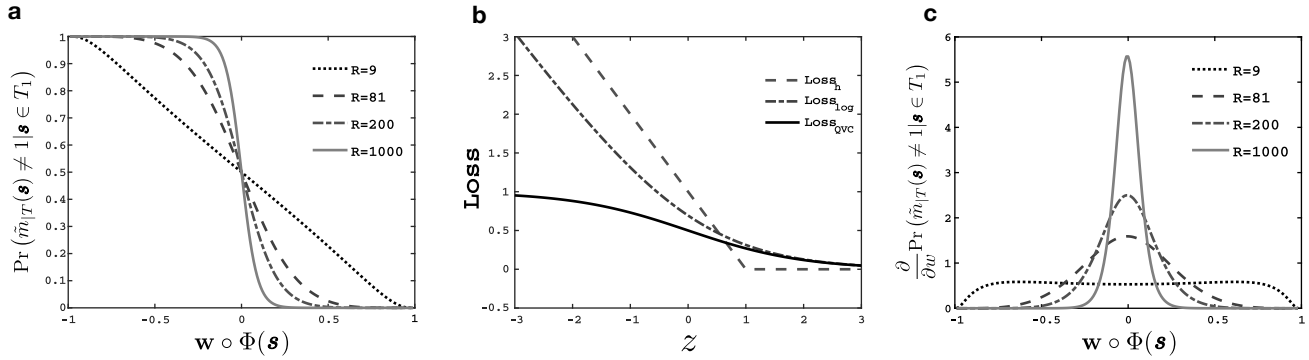

FIG. 4. (a) For a single shot (R=1) the assignment error of the decision rule is given by $1 - p_y$. The sigmoid contribution, eqn. (43) to the cost-function interpolates from linear to logistic-normal CDF (approximately sigmoid). In the experiment the data was sampled with 2000 shots, although the cost-function was evaluated with only $R = 200$ to provide a smoother function to the optimization routine. The plot depicts the cost-function as a function of the inner product of feature vector and separating hyperplane. (b) Comparison of different loss function that can be used in the optimization of linear threshold functions. We depict the hinge loss $\mathrm{Loss}_h(z)$, logistic regression $\mathrm{Loss}_{log}(z)$ and the reversed sigmoid cost-function $\mathrm{Loss}_{QVC}(z)$ used for the variational circuit classifier in the experiment. (c) The derivative of the sigmoid cost-function eqn. (43) contribution with respect to the inner product of feature vector and separating hyperplane

To better compare the quantum variational classification with support vector machines we will provide some additional background and highlight the similarities and differences in terms of the optimization problem. The goal of a classical support vector machine, c.f. section III A, is to optimize a linear threshold function in feature space so that

this hyperplane has the largest possible margin between the two labeled data sets. This is of course only possible when the data is in fact separable by a hyperplane in feature space. In this case, a direct formulation of this problem reads

$$\begin{aligned} \text{maximize} \quad & \gamma \\ \text{subject to:} \quad & \|\mathbf{w}\|_2^2 = 1 \\ & y_i\left(\mathbf{w} \circ \Phi(\boldsymbol{x}_i) + b\right) \geq \gamma \quad \forall i = 1, \ldots, t. \end{aligned} \tag{44}$$

The advantage of the conventional optimization problem for SVMs eqn. (5) is that it turns this non-convex optimization problem in to a convex optimization problem by considering a normal $\mathbf{w}$ that is variable in length. Moreover, if the data is not linearly separable, we have discussed a common trick to allow for outliers by constructing a soft margin SVM, c.f. eqn. (8) and introducing slack variable $\xi_i \geq 0$ with $i = 1 \ldots t$ to soften the constraints. Note, that such a soft margin SVM can be implemented by eliminating the slack variables and modifying the SVM optimization problem to [1]

$$\text{minimize} \quad L_P = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{j=1}^{t} \max(1 - y_j(\mathbf{w} \circ \Phi(\boldsymbol{x}_j) + b), 0). \tag{45}$$

The normal of $\mathbf{w}$ is now unconstrained, and the soft margin constraints are implemented through the hinge loss function $\text{Loss}_h(z) = \max(1 - z, 0)$. The term $\|\mathbf{w}\|_2^2$ is often referred to as the regularization and the total cost function is a sum of convex functions and therefore convex itself.

The optimization problem for the quantum variational classifier differs from this optimization problem in several points. First, note that both the non-linear dependence of the variational circuit family $W(\boldsymbol{\theta})$ on $\boldsymbol{\theta}$, as well as the constraint $\text{tr}\left[\left(W(\boldsymbol{\theta})^\dagger \mathbf{f} W(\boldsymbol{\theta})\right)^2\right] = 2^n$ (which is the same as demanding that $\|\mathbf{w}\|_2^2 = 1$ for the appropriate inner product) gives rise to a non-convex optimization problem. But more importantly, note that the label assignment $\tilde{m}(\boldsymbol{x})$ of the variational classifier is subject to shot noise even when a large number of samples R are used. This noise is present during the evaluation of the circuit for training and later during classification as well.

A common approach to training a linear classifier that produces noisy labels is through a logistic regression model. In the optimization of this model the hinge loss cost function is replaced by the logistic regression function that can be seen as the maximization of the log - likelihood with a quadratic regularization term [1]

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{j=1}^{t} \log(1 + e^{y_j(\mathbf{w} \circ \Phi(\boldsymbol{x}_j) + b)}). \tag{46}$$

Here, again the cost function is constructed from a quadratic regularizer and a logistic regression functions $\text{Loss}_{log}(z) = -\log(\text{sig}(z))$, where $\text{sig}(z)$ refers to the sigmoid function defined above. Note, that this logistic regression function can be seen as a smoothing of the hinge loss function used in the soft margin SVM as depicted in Fig 4.b.

The setting we consider is still different. We actually use the noisy labels to optimize the cost-function. While in the logistic regression model the evaluation of the cost function is noise free, we have a scenario where we optimize the classifier based on the output of the quantum circuit directly. So the labels that are observed during training are also subject to statistical uncertainty. This, and the fact that we consider a non-convex optimization over the parametrized hyper plane with unit normal constraint make the optimization problem different from the logistic regression model. As explained above, we motivate the choice of cost-function to minimize the average error of assigning a wrong label and substitute the empirical estimate $\hat{p}_y(\boldsymbol{x})$ in lieu of the actual probability $p_y(\boldsymbol{x})$ in eqn. (43) and optimize

$$\text{minimize} \quad \sum_{j=1}^{t} \text{sig}\left(\sqrt{R}\frac{\frac{1-y_j b}{2} - \hat{p}_{y_j}(\boldsymbol{x}_j)}{\sqrt{2(1-\hat{p}_{y_j}(\boldsymbol{x}_j))\hat{p}_{y_j}(\boldsymbol{x}_j)}}\right). \tag{47}$$

By construction we have that $0 \leq \hat{p}_y \leq 1$ and the optimization is over unitary circuits $W(\boldsymbol{\theta})$, which constraints the norm of the hyperplane normal to unity. Recall, c.f. eqn. (20), that the probability of observing the outcome y is given by $p_y(\boldsymbol{x}) = 2^{-1}(1 + y\langle\Phi_\alpha(\boldsymbol{x})\,|W^\dagger(\boldsymbol{\theta})\mathbf{f}W(\boldsymbol{\theta})|\,\Phi_\alpha(\boldsymbol{x})\rangle) = 2^{-1}(1 + y\ \boldsymbol{w} \circ \Phi(\boldsymbol{x}))$, which can be used to re-express the summands in the total cost -function eqn. (43) as

$$\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq y | \boldsymbol{s} \in T_y\right) \approx \text{sig}\left(-\frac{y\left(\boldsymbol{w} \circ \Phi(\boldsymbol{x}) + b\right)}{\sqrt{\frac{2}{R}(1 - (\boldsymbol{w} \circ \Phi(\boldsymbol{x}))^2)}}\right). \tag{48}$$

We have that $\mathrm{sig}(-z) = 1 - \mathrm{sig}(z)$ so that, taking in to account the normalization of the argument in terms of denominator that considers the statistical fluctuations at finite R, we are left with a loss function $\mathrm{Loss}_{QVC} = 1-\mathrm{sig}(z)$ that is compared to both the hinge loss and the logistic regression in Fig 4.b.

Note, that the cost function $\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq 1 | \boldsymbol{s} \in T_1\right)$ as depicted in Fig 4.a is most sensitive to states for which $\boldsymbol{w} \circ \Phi(\boldsymbol{x})$ is small. These are states that lie close to the decision boundary. When we perform a gradient based optimization, we see that the largest contribution to the gradient comes from the states that lie close to this boundary, Fig 4.c. The cost function therefore improves mostly by increasing the distance of the Hyperplane relative to the close to the boundary, which can be interpreted as increasing the margin of the linear separator. Note, that the width of this region depends on the parameter R. While the single shot case (for which $R = 1$) yields a cost-function $\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq y | \boldsymbol{s} \in T_y\right) = 1 - p_y(\boldsymbol{s})$ that is sensitive to all points in the training set, we have that as R increases the contributions to the gradient concentrate increasingly at the decision boundary Fig 4.c. This behavior lends itself to the interpretation that, although the optimization considered here is not the conventional SVM problem, its objective of increasing the distance to states that lie close to the decision boundary is in fact similar.

There are many other cost functions that can be considered to train the classifier. Examples range from hinge loss, over logistic - regression, the perceptron criterion to the squared loss and others. We have chosen the cost-function implemented in the experiment based on the statistical analysis of the histogram decision criterion discussed above. Furthermore it has the property of focusing on points that lie close to the decision boundary.

B. Multi label classification

For multiple labels, one tries to optimize

$$P_{err} = \frac{1}{|T|} \sum_c \sum_{\boldsymbol{s} \in T_c} \Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq c | \boldsymbol{s} \in T_c\right) ,$$

where:

$$\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq c | \boldsymbol{s} \in T_c\right) = \Pr\left(n_c < \max_{c'}\left(\{n_{c'}\}_{c'/c}\right)\right) .$$

We consider the case of three labels. For R samples with frequencies $\{n_0, n_1, n_2\}$, drawn independently from the output probability distribution, the probability of misclassifying a sample $s \in T_0$ by argmax is given by

$$\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq 0 | \boldsymbol{s} \in T_0\right) = \Pr\Big(n_0 < \max(n_1, n_2)\Big) = \Pr\left(n_0 < \left\lceil \frac{N + |n_1 - n_2|}{3} \right\rceil\right) ,$$

where the last inequality is derived as follows

$$2n_0 < 2\max(n_1, n_2) = |n_1 - n_2| + n_1 + n_2 = |n_1 - n_2| + N - n_0 .$$

Hence setting $\gamma = \frac{N+|n_1-n_2|}{3}$, it follows that

$$\Pr\left(\tilde{m}_{|T}(s) \neq 0 | s \in T_0\right) = \sum_{k=0}^{k=\gamma} \binom{R}{k} p_0^k (1-p_0)^{N-k} \approx \mathrm{sig}\left(\frac{\gamma - Np_0}{\sqrt{2N(1-p_0)p_0}}\right) .$$

This however still depends on n_1, n_2, which can't be simply eliminated. Additionally, for a general k-label case, there is no simple analytic solution for γ. For this reason, we try to estimate the above probability by making the approximation $\gamma = \max_{c'}\left(\{n_{c'}\}_{c'/c}\right)$. So for k-label case, the cost function terms are given by

$$\Pr\left(\tilde{m}_{|T}(\boldsymbol{s}) \neq c | \boldsymbol{s} \in T_c\right) \approx \mathrm{sig}\left(\sqrt{R}\frac{\max_{c'}\left(\{n_{c'}\}_{c'/c}\right) - n_c}{\sqrt{2(N-n_c)n_c}}\right) .$$

VII. QUANTUM KERNEL ESTIMATION

For the second classification method we only use the quantum computer to estimate the kernel $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = |\langle \Phi(\boldsymbol{x}_i) | \Phi(\boldsymbol{x}_j) \rangle|^2$. Then we use the classical optimization problem as outlined again in eqn. (10) to obtain the

optimal Lagrange multipliers α_i and support vectors N_S can be obtained. From this the classifier can be constructed, c.f .eqn. (19). To apply the classifier to a new datum $\boldsymbol{s} \in S$ the kernel $K(\boldsymbol{x}_i, \boldsymbol{s})$ between $\boldsymbol{s}$ and the support vectors in $i \in N_S$ has to be estimated. We discuss two methods to estimate this overlap.

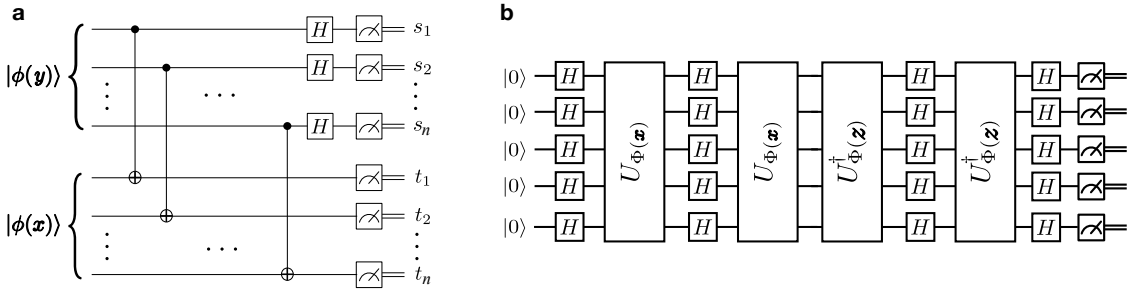

FIG. 5. a) Estimating the expectation value of the SWAP- matrix as derived in [14]. This circuit diagonalizes the SWAP - gate, when it is treated as a hermitian observables with eigenvalues ± 1. Averaging the eigenvalues, c.f. eqn. (49), with samples from the output distribution constructs an estimator for $|\langle \Phi(\boldsymbol{x}) | \Phi(\boldsymbol{y}) \rangle|^2$. b) The circuit estimates the fidelity between two states in feature space directly by first applying the unitary $\mathcal{U}_{\Phi(\boldsymbol{x})}$ followed by the inverse $\mathcal{U}^\dagger_{\Phi(\boldsymbol{y})}$ and measuring all bits at the output of the circuit. The frequency $\nu_{0,\ldots,0} = |\langle \Phi(\boldsymbol{x}) | \Phi(\boldsymbol{y}) \rangle|^2$ of the all the zero outcome precisely corresponds to the desired state overlap.

The usual method of estimating the fidelity between two states is by using the swap test [15]. This circuit, however, is not a short depth circuit on a quantum computing architecture with geometrically local gates. It would require a sequence of controlled SWAP, also known as Fredkin gates, all conditioned on the state of the same ancilla qubit. A very nice protocol was recently developed in [14]. The authors have learned multiple ways of optimizing the conventional swap test. If only the value of the fidelity is needed, as is the case for our algorithm, the authors propose, c.f. [14] section III.C, a circuit that is constant depth if pairs of CNOT gates can be executed in parallel. This circuit, c.f. Fig 5.a, evaluates the expectation value $\langle \psi | \langle \phi | \mathsf{SWAP} | \psi \rangle | \phi \rangle$ directly. The action of their algorithm can be understood as follows:

The SWAP gate is both a unitary gate and a hermitian observable $\mathsf{SWAP}^\dagger = \mathsf{SWAP}$ with eigenvalues ± 1. The expectation value on two product states as we said given by $\langle \psi | \langle \phi | \mathsf{SWAP} | \psi \rangle | \phi \rangle = |\langle \phi | \psi \rangle|^2$. Now the gate can be decomposed in to a product of two qubit swap gates $\mathsf{SWAP} = \prod_{k=1}^n \mathsf{SWAP}_{s_k t_k}$ that act all in parallel. To evaluate the expectation value one has to diagonalize the full gate. This is achieved by diagonalizing the individual two-qubit swap gates and observing that $\mathsf{SWAP}_{ij} = \mathsf{CNOT}_{i\to j}\mathsf{CNOT}_{j\to i}\mathsf{CNOT}_{i\to j}$. Using the circuit identity $\mathsf{CNOT}_{j\to i} = H_j \mathsf{CZ}_{ji} H_j$, we see that SWAP_{ij} is diagonalized by $\mathsf{CNOT}_{j\to i} H_j$ and has eigenvalue $(-1)^{x_i x_j}$. For the full circuit Fig 5.a one first applies a transversal set of CNOT gates across both registers followed by a single layer of Hadamard gates H on the top register. Then the output is sampled and the average of the boolean function

$$f(s,t) = (-1)^{(s_1 t_1 + \ldots s_n t_n)} \tag{49}$$

is reported. The output bits on the top register are labeled by $s \in \{0,1\}^n$, while $t \in \{0,1\}^n$ is the output string on the lower register.

This method works for arbitrary input states $|\psi\rangle, |\phi\rangle$. Our states, however, are structured and are all generated by the unitary eqn. (31) as shown in Fig 2. Writing out the kernel explicitly $K(\boldsymbol{y}, \boldsymbol{x}) = |\langle \Phi(\boldsymbol{y}) | \Phi(\boldsymbol{x}) \rangle|^2 = |\langle 0^n | \mathcal{U}^\dagger_{\Phi(\boldsymbol{y})} \mathcal{U}_{\Phi(\boldsymbol{x})} | 0^n \rangle|^2$ gives the indication of how to measure it. Simply apply the circuit $\mathcal{U}^\dagger_{\Phi(\boldsymbol{y})} \mathcal{U}_{\Phi(\boldsymbol{x})}$ to the state $|0^n\rangle$, c.f. Fig 5.b. Then sample R times the resulting state $\mathcal{U}^\dagger_{\Phi(\boldsymbol{y})} \mathcal{U}_{\Phi(\boldsymbol{x})} | 0^n \rangle$ in the Z basis. Record the number r_{0^n} of all observed zero $0^n = (0, \ldots, 0)$ bit-strings and divide by the total number of shots R. The frequency $\mathrm{r}_{0^n} R^{-1}$ then gives an estimator for $K(\boldsymbol{y}, \boldsymbol{x})$ up to a sampling error $\tilde{\epsilon} = \mathcal{O}(R^{-1/2})$. A rough estimate for the operator norm $\|\cdot\|$ of the deviation of the resulting estimator $\hat{K}$ from the true kernel matrix K is given by $\|K - \hat{K}\| \leq \|K - \hat{K}\|_F$. Here $\|A\|_F = \sqrt{\sum_{ij} |A_{ij}|^2}$ denotes the Frobenius norm of the matrix A. A crude bound can then be obtained from the largest sampling error $\tilde{\epsilon}$ of all matrix entries and setting $\|K - \hat{K}\|_F \leq \tilde{\epsilon}|T|$, since both matrices of the training set T are of size $|T| \times |T|$. Hence to ensure a maximum deviation of ϵ with high probability a number of $R = \mathcal{O}(\epsilon^{-2}|T|^2)$ shots have to be drawn for each matrix entry. Due to the symmetry of the

K matrix and the trivial diagonals, $|T|(|T|-1)2^{-1}$ matrix entries have to be estimated. Thus the full sampling complexity is expected to scale as $\mathcal{O}(\epsilon^{-2}|T|^4)$. A more careful analysis of the statistical error could be carried out by using one of the matrix-concentration results [16].

Note that the optimization problem, eqn. (10) is only concave, when the matrix $K \geq 0$ is positive semi-definite. It can happen, that the shot noise and other errors in the experiment lead to a $\hat{K}$ that is no longer positive semi-definite. We have indeed observed this multiple times in the experiment. A possible way of dealing with this problem is a method developed in [17], where an optimization problem is solved to find the closest positive semi-definite K-matrix in trace norm to $\hat{K}$ consistent with the constraint. In our experiments however, we have found this not to be necessary and the performance has been almost optimal without performing this method.

VIII. DEVICE PARAMETERS

Our experimental device consists of five coupled superconducting transmons, only two of which are used in this work. Two co-planar waveguide (CPW) resonators, acting as quantum buses, provide the device connectivity. Each qubit has one additional CPW resonator for control and readout. Entanglement in our system is achieved via CNOT gates, which use cross-resonance [18] as well as single qubit gates as primitives. The quantum processor is thermally anchored to the mixing chamber plate of a dilution refrigerator.

Our device is fabricated on a 720-μm-thick Si substrate. A single optical lithography step is used to define all CPW structures and the qubit capacitors with Nb. The Josephson junctions are patterned via electron beam lithography and made by double-angle deposition of Al.

The dispersive readout signals are amplified by Josephson Parametric Converters [19] (JPC). Both the quantum processor and the JPC amplifiers are thermally anchored to the mixing chamber plate of a dilution refrigerator.

The two qubit fundamental transition frequencies are $\omega_i/2\pi = \{5.2760(4), 5.2122(3)\}$ GHz, with anharmonicities $\Delta_i/2\pi = \{-330.3, -331.9\}$ MHz, where $i \in \{0, 1\}$. The readout resonator frequencies used are $\omega_{Ri}/2\pi = \{6.530553, 6.481651\}$ GHz, while the CPW bus resonator connecting Q_0 and Q_1 was unmeasured and designed to be 7.0 GHz. The dispersive shifts and line widths of the readout resonators are measured to be $2\chi_i/2\pi = \{-1.06, -1.02\}$ MHz and $\kappa_i/2\pi = \{661, 681\}$ kHz, respectively.

The two qubit lifetimes and coherences were measured intermittently throughout our experiments. The observed mean values were $T_{1(i)} = \{55, 38\}$, $T^*_{2(i)} = \{16, 17\}$, $T^{\text{echo}}_{2(i)} = \{43, 46\}$ μs with $i \in \{0, 1\}$

A. Gate characterization

Our experiments use calibrated $X-$rotations (X_π and $X_{\pi/2}$) as single-qubit unitary primitives. $Y-$rotations are attained by appropriate adjustment of the pulse phases, whereas $Z-$rotations are implemented via frame changes in software [20]. A time buffer is added at the end of each physical pulse to mitigate effects from reflections and evanescent waves in the cryogenic control lines and components.

We use two sets of gate times in order to perform the Richardson extrapolation of the noise in our system [21, 22]. For the first set of gate times we use 83 ns for single-qubit gates and 333 ns for each cross-resonance pulse. The buffer after each physical pulse is 6.5 ns. The single-qubit gates are gaussian shaped pulses with $\sigma = 20.75$ ns. The cross-resonance gates are flat with turn-on and -off gaussian shapes of $\sigma = 10$ ns. Our implementation of a CNOT has a duration of two single-qubit pulses and two cross-resonance pulses, giving a total of 858 ns for the first set of gate times, including buffers. For our second set of gate times we use the times of the first set but stretched by a factor of 1.5, including the pulses σs and the buffers. This gives a total CNOT time of 1.287 μs and single-qubit gates of ~ 125 ns.

We experimentally verified our single- and two-qubit unitaries by Randomized Benchmarking (RB) [23, 24]. The following table shows the RB results for all single-qubit gates used in our experiments, including individual and simultaneous RB.

Qubit label	Q_0 (83 ns) ($\times 10^{-3}$)	Q_1 (83 ns) ($\times 10^{-3}$)	Q_0 (125 ns) ($\times 10^{-3}$)	Q_1 (125 ns) ($\times 10^{-3}$)
01	-	0.715 ± 0.005	-	1.244 ± 0.010
10	1.319 ± 0.017	-	1.410 ± 0.010	-
11	1.367 ± 0.011	0.763 ± 0.005	1.484 ± 0.014	1.271 ± 0.010

TABLE I. RB of our single-qubit gates. Qubit labels indicate which qubit was benchmarked on each case, with label 11 indicating simultaneous RB.

Our two-qubit unitaries are CNOTs constructed from echo cross-resonance sequences [24, 25]. Each of the two cross-resonance pulses in a CNOT has durations of 333 and 500 ns for the two different gate lengths used in our experiments. For our two-qubit RB we obtain a CNOT error of $0.0373 \pm .0015$ ($0.0636 \pm .0021$) for the 333 (500) ns cross-resonance pulse.

B. Readout correction

Our readout assigned fidelity was $\sim 95\%$ for both qubits.

For each experiment, we run 4 (2^2) calibration sequences preparing our two qubits in their joint computational states. We gather statistics of these calibrations and create a measurement matrix $A_{ij} = P(|i\rangle||j\rangle)$ where $P(|n\rangle||m\rangle)$ is the probability of measuring state $|m\rangle$ having prepared state $|n\rangle$. We then correct the observed outcomes of our experiments by inverting this matrix and multiplying our output probability distributions by this inverse.

IX. ERROR MITIGATION AND ADDITIONAL EXPERIMENTAL RESULTS

The experimental estimation of the kernel matrices shown in Fig. 9 and in Fig. 4 in the main text involves running the experiments at different gate lengths and extrapolating the expectation value of the observable of interest to its zero-noise value. The technique [21, 22] is extremely powerful in coping with incoherent errors, at the cost of increased sampling and without requiring any additional quantum resources. However, the experimental readout assignment fidelities also determine how precisely an expectation value can be estimated.

Even though for our Sets I and II we attain 100 % classification success over 10 randomly drawn test sets in each case, we can quantify how close our experimentally determined separating hyperplane is to the ideal.

The optimal hyperplane for a given training set can be expressed as the linear combination $\sum_i \alpha_i y_i \Phi(\boldsymbol{x}_i) = \mathbf{w}$ (eqn. 13), where $\mathbf{w}$ is a vector orthogonal to the optimal separating hyperplane and $\Phi(\boldsymbol{x}_i)$ are the support vectors. We can therefore quantify the distance between the experimentally obtained hyperplane and the ideal hyperplane by calculating the inner product $\langle \mathbf{w}, \mathbf{w}_{\text{ideal}} \rangle = \sum_{i \in N_S} \sum_{j \in N'_S} y_i y_j \alpha_i^* \alpha_j |\langle \boldsymbol{x}_i \boldsymbol{x}_j \rangle|^2 / ||\mathbf{w}|| ||\mathbf{w}_{\text{ideal}}||$ where N_S and N'_S are the sets of experimentally obtained and ideal support vectors, respectively.

In Fig. 6 we show the inner products between the ideal and all experimental hyperplanes, including the two sets of gate times used throughout our experiments, $c1$ and $c1.5$, as well as the error-mitigated hyperplanes.

For Sets I and II, which classify at 100 % success, it is clear that error mitigation improves our results very significantly. This is not the case for Set III, which classifies at 94.75 % success. In fact, for Set III we see that error-mitigation worsens the hyperplane, as the results are closer to the ideal for the unmitigated experiments than both Sets I and II.
A look at the calibration data taken along the direct kernel estimation experiments for each set, we see that the readout assignment fidelities of $Q_0(Q_1)$ are 96.56% (96.31%) for Set I, 95.90% (96.36%) for Set II, and 93.99% (95.47%) for Set III. The slightly worse readout fidelities for Set III could partially explain the worse classification results for this set. However, one can not discount other experimental imperfections such as finite sampling effects and gate tune - up errors.

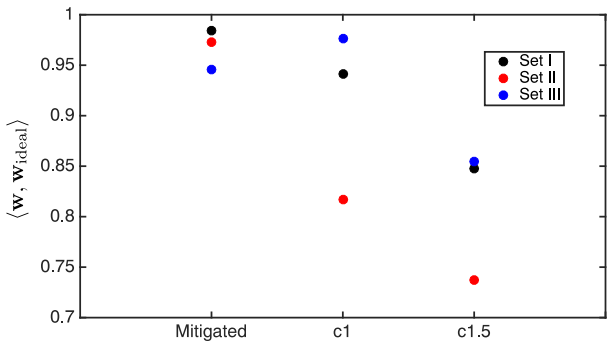

FIG. 6. Inner products between experimentally obtained hyperplanes and the ideal for each training set. The $x-$axis shows the results for the two different time stretches used in our experiments, $c1$ and $c1.5$, corresponding to the faster and slower gates, respectively. We also show the error-mitigated hyperplanes inner products.

A. Results for additional data sets

Here we show the support vectors and α_i as calculated for each of the three datasets studied from their K matrices.

Set I		
SV	α	y
[5.3407,2.9531]	2.4185	+1
[5.1522,4.6496]	2.6248	+1
[0.6912,0.7540]	3.1379	+1
[6.0947,5.7177]	1.2891	+1
[6.0947,4.1469]	1.3604	+1
[2.6389,4.8381]	1.7163	+1
[4.1469,2.0735]	1.3242	-1
[0.2513,3.2044]	0.9757	-1
[2.3248,3.9584]	1.4953	-1
[4.1469,6.2204]	2.4536	-1
[2.6389,1.6965]	0.5293	-1
[5.4664,4.6496]	4.0213	-1
[4.1469,3.0788]	1.7477	-1
bias = -0.1090		

Set II		
SV	α	y
[0.1885,3.3301]	2.1871	+1
[5.4035,1.7593]	1.443	+1
[5.2150,3.7071]	2.0330	+1
[4.71240,4.6496]	0.7559	+1
[0.4398,2.388]	1.0022	+1
[6.2204,5.7177]	2.3090	+1
[5.9062,4.2726]	1.6297	-1
[1.8221,2.1362]	2.9281	-1
[0.3770,3.8327]	3.0627	-1
[0.31416,1.6965]	1.3793	-1
[1.6336,3.2672]	0.2759	-1
[2.5761,3.2673]	0.4544	-1
bias = 0.2102		

Set III		
SV	α	y
[5.4664,3.7071]	6.2649	+1
[1.5080,5.2779]	0.7777	+1
[5.9690,3.8956]	1.5619	+1
[5.3407,3.5186]	0.6482	+1
[5.7177,4.5239]	1.6387	+1
[0.0628,3.5186]	0.9379	+1
[6.1575,1.445]	0.3882	-1
[5.9690,0.9425]	0.8346	-1
[0.2513,1.3823]	2.5561	-1
[5.2779,2.0735]	4.8076	-1
[6.0947,2.0106]	3.2428	-1
bias = -0.1865		

TABLE II. Suport vectors for all three data sets used for our kernel estimation method, as shown as green circles in Figs. 3(b) (main text) and Fig. 7.

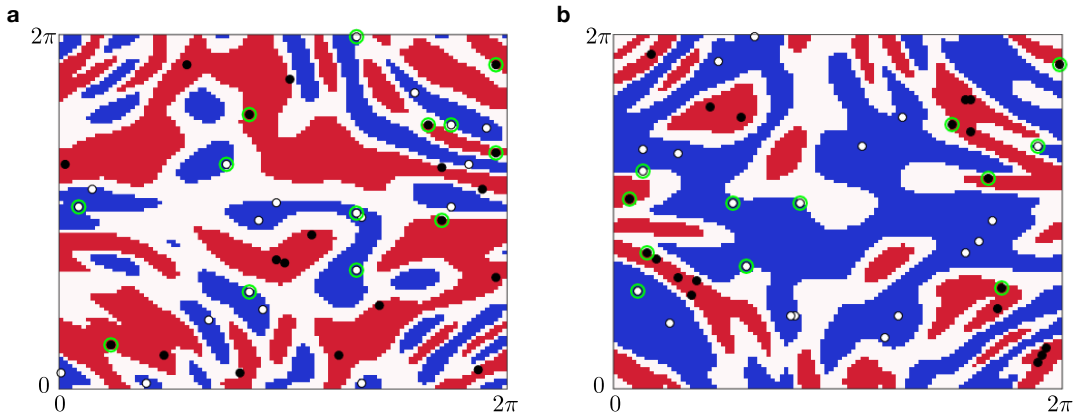

FIG. 7. Sets I (a) and II (b), including training data points (white and black circles) and support vectors (green circles).

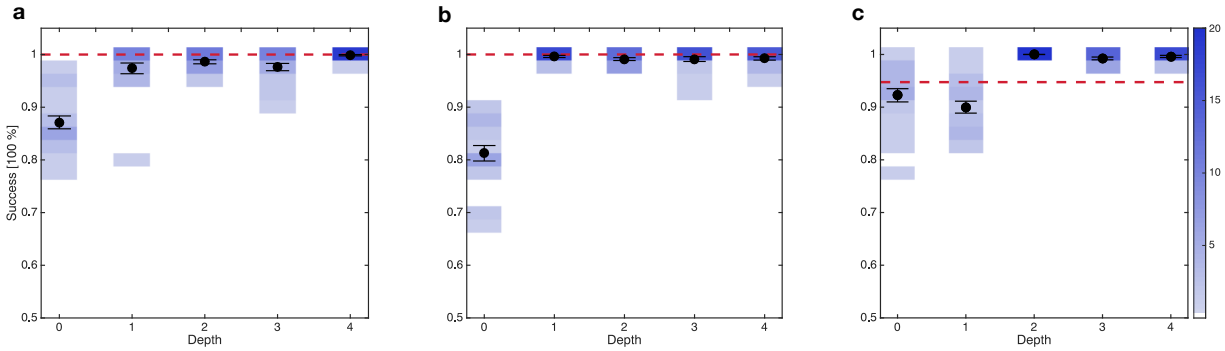

FIG. 8. Classification results for the quantum variational method for Sets I (a), II (b), and III (c) as a function of circuit depth. For each set and each depth a total of 20 test sets consistings of 20 data points per lable are randomly drawn and classified. The black dots are the mean value of the classification successes for each set at each depth, with the standard error of the mean given by the error bars. Each depth also contains a density plot showing the histograms for all 20 test sets classified. The width of each histogram bin corresponds to the precision of our success gradation and is equal to $1/40 = 0.025$. The combined results for the three plots are shown in the main text (Fig. 2c). The red dashed lines show the average success results for each set of the direct kernel method, for comparison.

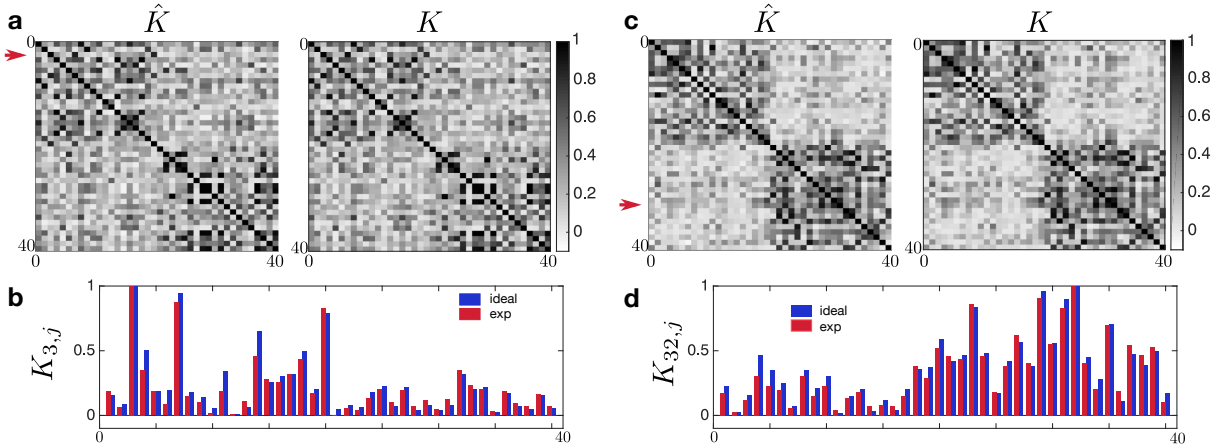

FIG. 9. Experimentally estimated and ideal kernel matrices for Sets I (a, b) and II (c, d). For both datasets we show a cut (b, d) through the row at which the maximum of $|\hat{K} - K|$ occurs, similarly as in Fig. 4 in the main text.

[1] Vapnik, V. *The nature of statistical learning theory* (Springer science & business media, 2013).

[2] Burges, C. J. A tutorial on support vector machines for pattern recognition. *Data mining and knowledge discovery* **2**, 121–167 (1998).

[3] Murty, K. G. & Yu, F.-T. *Linear complementarity, linear and nonlinear programming*, vol. 3 (Citeseer, 1988).

[4] Boser, B. E., Guyon, I. M. & Vapnik, V. N. A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, 144–152 (ACM, 1992).

[5] Stoudenmire, E. & Schwab, D. J. Supervised learning with tensor networks. In *Advances in Neural Information Processing Systems*, 4799–4807 (2016).

[6] Van Dam, W., Hallgren, S. & Ip, L. Quantum algorithms for some hidden shift problems. *SIAM Journal on Computing* **36**, 763–778 (2006).

[7] Rötteler, M. Quantum algorithms for highly non-linear boolean functions. In *Proceedings of the twenty-first annual ACM-SIAM symposium on Discrete algorithms*, 448–457 (Society for Industrial and Applied Mathematics, 2010).

[8] Aaronson, S. & Ambainis, A. Forrelation: A problem that optimally separates quantum from classical computing. *SIAM Journal on Computing* **47**, 982–1038 (2018).

[9] Bremner, M. J., Montanaro, A. & Shepherd, D. J. Average-case complexity versus approximate simulation of commuting quantum computations. *Physical review letters* **117**, 080501 (2016).

[10] Goldberg, L. A. & Guo, H. The complexity of approximating complex-valued ising and tutte partition functions. *computational complexity* **26**, 765–833 (2017).

[11] Kandala, A. *et al.* Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature* **549**, 242 (2017).

[12] Farhi, E., Goldstone, J., Gutmann, S. & Neven, H. Quantum algorithms for fixed qubit architectures. *arXiv preprint arXiv:1703.06199* (2017).

[13] d'Alessandro, D. *Introduction to quantum control and dynamics* (CRC press, 2007).

[14] Cincio, L., Subaşı, Y., Sornborger, A. T. & Coles, P. J. Learning the quantum algorithm for state overlap. *arXiv preprint arXiv:1803.04114* (2018).

[15] Buhrman, H., Cleve, R., Watrous, J. & De Wolf, R. Quantum fingerprinting. *Physical Review Letters* **87**, 167902 (2001).

[16] Tropp, J. A. *et al.* An introduction to matrix concentration inequalities. *Foundations and Trends ® in Machine Learning* **8**, 1–230 (2015).

[17] Smolin, J. A., Gambetta, J. M. & Smith, G. Efficient method for computing the maximum-likelihood quantum state from measurements with additive gaussian noise. *Physical review letters* **108**, 070502 (2012).

[18] Rigetti, C. & Devoret, M. Fully microwave-tunable universal gates in superconducting qubits with linear couplings and fixed transition frequencies. *Phys. Rev. B* **81**, 134507 (2010). URL https://link.aps.org/doi/10.1103/PhysRevB.81.134507.

[19] Bergeal, N. *et al.* Analog information processing at the quantum limit with a josephson ring modulator. *Nature Physics* **6**, 296 EP – (2010). URL http://dx.doi.org/10.1038/nphys1516. Article.

[20] McKay, D. C., Wood, C. J., Sheldon, S., Chow, J. M. & Gambetta, J. M. Efficient z gates for quantum computing. *Physical Review A* **96**, 022330 (2017).

[21] Temme, K., Bravyi, S. & Gambetta, J. M. Error mitigation for short-depth quantum circuits. *Physical review letters* **119**, 180509 (2017).

[22] Kandala, A. *et al.* Extending the computational reach of a noisy superconducting quantum processor. *arXiv preprint arXiv:1805.04492* (2018).

[23] Gambetta, J. M. *et al.* Characterization of addressability by simultaneous randomized benchmarking. *Phys. Rev. Lett.* **109**, 240504 (2012). URL https://link.aps.org/doi/10.1103/PhysRevLett.109.240504.

[24] Córcoles, A. D. *et al.* Process verification of two-qubit quantum gates by randomized benchmarking. *Phys. Rev. A* **87**, 030301 (2013). URL https://link.aps.org/doi/10.1103/PhysRevA.87.030301.

[25] Sheldon, S., Magesan, E., Chow, J. M. & Gambetta, J. M. Procedure for systematically tuning up cross-talk in the cross-resonance gate. *Phys. Rev. A* **93**, 060302 (2016).